Kubernetes: Хранилища данных и конфигурации для PostgreSQL DBA

Дополнительный материал для курса PostgreSQL Advanced

Дмитрий Золотов

2026-02-13

1 Kubernetes: Хранилища данных и конфигурации для PostgreSQL DBA
1.1 Оглавление
1.2 ConfigMaps: управление конфигурациями
1.2.1 Мотивация: проблема хардкода
1.2.2 Решение: ConfigMap
1.2.3 Создание ConfigMap
1.2.4 Использование ConfigMap в Pod
1.2.5 Горячее обновление ConfigMap
1.2.6 Best Practices для ConfigMaps
1.3 Secrets: безопасное хранение паролей и ключей
1.3.1 Мотивация: проблема хранения паролей
1.3.2 Решение: Secrets
1.3.3 Создание Secret
1.3.4 Использование Secrets в Pod
1.3.5 Специальные типы Secrets
1.3.6 Безопасное управление Secrets в Production
1.3.7 Ротация Secrets
1.3.8 Best Practices для Secrets
1.4 Volumes: основы хранения данных
1.4.1 Мотивация: проблема эфемерности контейнеров
1.4.2 Решение: Volumes
1.4.3 Типы Volumes

1.5 Persistent Volumes: долговечное хранилище

1.5.1 Архитектура PV и PVC

1.5.2 Создание Persistent Volume

1.5.3 Access Modes

1.5.4 Reclaim Policy

1.6 StatefulSets: для stateful приложений

1.6.1 Почему Deployment не подходит для баз данных

1.6.2 Решение: StatefulSet

1.6.3 Пример StatefulSet для PostgreSQL

1.6.4 DNS-имена для StatefulSet

1.6.5 Упорядоченный запуск и остановка

1.6.6 Update Strategy

1.7 Storage Classes: динамическое выделение хранилища

1.7.1 Проблема статического выделения

1.7.2 Решение: StorageClass

1.7.3 Пример: AWS EBS StorageClass

1.7.4 Volume Binding Modes

1.7.5 Расширение дисков (Volume Expansion)

1.7.6 Снапшоты (VolumeSnapshot)

1.8 Практика: PostgreSQL с PVC

1.8.1 Полный пример: PostgreSQL StatefulSet в AWS EKS

1.9 Helm: пакетный менеджер Kubernetes

1.9.1 Зачем нужен Helm

1.9.2 Helm: решение

1.9.3 Установка Helm

1.9.4 Использование готовых Charts

1.9.5 Создание своего Chart

1.10 PostgreSQL Operators: автоматизация управления

1.10.1 Проблема ручного управления

1.10.2 Решение: Kubernetes Operators

1.10.3 Три главных PostgreSQL Operator

1.10.4 Сравнение: Вручную vs Helm vs Operator

1.11 Troubleshooting и best practices

1.11.1 Проблема 1: Pod не может примонтировать PVC

1.11.2 Проблема 2: PostgreSQL не запускается (PGDATA not empty)

1.11.3 Проблема 3: Secret не обновляется в поде

1.11.4 Проблема 4: PVC застрял в статусе Pending

1.11.5 Проблема 5: Медленная работа базы данных

1.11.6 Best Practices

1.12 Полезные ресурсы

1.12.1 Официальная документация

1.12.2 PostgreSQL Operators

1.12.3 Helm Charts

1.12.4 Инструменты

1.12.5 Книги и курсы

1.12.6 Блоги и статьи

1.13 Заключение

1.13.1 Что вы теперь знаете

1.13.2 Ключевые выводы

1.13.3 Что дальше

1 Kubernetes: Хранилища данных и конфигурации для PostgreSQL DBA

Дополнительный материал к курсу PostgreSQL Advanced Автор: Дмитрий Золотов

1.1 Оглавление

1. ConfigMaps: управление конфигурациями
2. Secrets: безопасное хранение паролей и ключей
3. Volumes: основы хранения данных
4. Persistent Volumes: долговечное хранилище
5. StatefulSets: для stateful приложений
6. Storage Classes: динамическое выделение хранилища
7. Практика: PostgreSQL с PVC
8. Helm: пакетный менеджер Kubernetes
9. PostgreSQL Operators: автоматизация управления
10. Troubleshooting и best practices
11. Полезные ресурсы

1.2 ConfigMaps: управление конфигурациями

1.2.1 Мотивация: проблема хардкода

Типичная ошибка:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    env:
    - name: POSTGRES_DB
      value: "myapp"  # ❌ Хардкод!
    - name: POSTGRES_USER
      value: "postgres"  # ❌ Хардкод!
    - name: MAX_CONNECTIONS
      value: "100"  # ❌ Хардкод!
```

Проблемы:

- ❌ Конфигурация разных окружений (dev/staging/prod) требует изменения манифестов
- ❌ Невозможно переиспользовать один манифест с разными параметрами
- ❌ Конфигурация распределена по множеству YAML-файлов
- ❌ Изменение параметра требует пересборки образа или редактирования манифеста

1.2.2 Решение: ConfigMap

ConfigMap — это объект Kubernetes для хранения конфигурационных данных в формате ключ-значение. Отделяет конфигурацию от кода приложения.

Аналогия для DBA:

ConfigMap — это как файл `postgresql.conf`, но:

- Хранится в etcd (как централизованный Config Server)
- Может быть примонтирован как том или использован как environment variables
- Версионируется и управляется через Kubernetes API
- Может обновляться без перезапуска подов (для Volume-режима)

1.2.3 Создание ConfigMap

Способ 1: Через YAML манифест

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-config
  namespace: default
data:
  # Простые пары ключ-значение
  POSTGRES_DB: "myapp"
  POSTGRES_MAX_CONNECTIONS: "200"
  POSTGRES_SHARED_BUFFERS: "256MB"

  # Многострочные значения (конфигурационные файлы)
  custom.conf: |
    # Custom PostgreSQL configuration
    shared_preload_libraries = 'pg_stat_statements'
    log_min_duration_statement = 1000
    log_connections = on
    log_disconnections = on

  pg_hba.conf: |
    # TYPE  DATABASE        USER            ADDRESS                 METHOD
    local   all             all                                     trust
    host    all             all             0.0.0.0/0               md5
    host    all             all             ::/0                    md5
```

Применение:

```bash
kubectl apply -f postgres-config.yaml

# Проверка
kubectl get configmaps
kubectl describe configmap postgres-config

# Просмотр содержимого
kubectl get configmap postgres-config -o yaml
```

Способ 2: Через kubectl create

```
# Из литералов
kubectl create configmap postgres-config \
  --from-literal=POSTGRES_DB=myapp \
  --from-literal=POSTGRES_MAX_CONNECTIONS=200

# Из файла
echo "shared_buffers = 256MB" > custom.conf
kubectl create configmap postgres-config \
  --from-file=custom.conf

# Из директории (все файлы станут ключами)
mkdir configs/
echo "log_min_duration_statement = 1000" > configs/logging.conf
echo "max_connections = 200" > configs/connections.conf
kubectl create configmap postgres-config --from-file=configs/
```

1.2.4 Использование ConfigMap в Pod

Способ 1: Environment Variables (envFrom)

Все ключи ConfigMap становятся environment variables:

```
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    envFrom:
    - configMapRef:
        name: postgres-config  # Все ключи → env vars
```

Результат внутри контейнера:

```
env | grep POSTGRES
# POSTGRES_DB=myapp
# POSTGRES_MAX_CONNECTIONS=200
# POSTGRES_SHARED_BUFFERS=256MB
```

Способ 2: Отдельные переменные (valueFrom)

Выборочное использование ключей:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    env:
    - name: POSTGRES_DB
      valueFrom:
        configMapKeyRef:
          name: postgres-config
          key: POSTGRES_DB
    - name: PGDATA
      value: /var/lib/postgresql/data/pgdata
```

Способ 3: Volume (монтирование файлов)

ConfigMap становится файлами внутри пода:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    volumeMounts:
    - name: config-volume
      mountPath: /etc/postgresql/custom.conf
      subPath: custom.conf  # Монтируем только один файл
    - name: config-volume
      mountPath: /etc/postgresql/pg_hba.conf
      subPath: pg_hba.conf
  volumes:
  - name: config-volume
    configMap:
      name: postgres-config
```

Внутри контейнера:

```
ls -l /etc/postgresql/
# -rw-r--r-- 1 root root 123 Feb 13 10:00 custom.conf
# -rw-r--r-- 1 root root 456 Feb 13 10:00 pg_hba.conf

cat /etc/postgresql/custom.conf
# shared_preload_libraries = 'pg_stat_statements'
# log_min_duration_statement = 1000
# ...
```

1.2.5 Горячее обновление ConfigMap

⚠️ **Важно:** Поведение зависит от способа использования:

Способ	Автоматическое обновление	Требуется рестарт пода
envFrom / valueFrom	❌ Нет	✅ Да
Volume	✅ Да (через ~60 сек)	⚠️ Зависит от приложения

Пример горячего обновления:

```
# Изменить ConfigMap
kubectl edit configmap postgres-config

# Или через patch
kubectl patch configmap postgres-config \
  --type merge \
  -p '{"data":{"POSTGRES_MAX_CONNECTIONS":"300"}}'

# Для Volume: файл обновится автоматически через ~60 секунд
# Для envFrom: нужен перезапуск пода
kubectl rollout restart deployment postgres-deployment
```

Для PostgreSQL: После обновления `custom.conf` через Volume:

```
# Внутри пода выполнить reload конфигурации
kubectl exec -it postgres-pod -- psql -U postgres -c "SELECT pg_reload_conf();"
```

1.2.6 Best Practices для ConfigMaps

✅ **Делайте:**

- Используйте отдельные ConfigMap для разных окружений (dev-config, prod-config)
- Версионируйте ConfigMap через Git
- Используйте Volume для конфигурационных файлов, которые могут обновляться

- Документируйте назначение каждого ключа

❌ Не делайте:

- Не храните пароли и секреты в ConfigMap (используйте Secrets)
- Не используйте огромные ConfigMap (> 1 MB)
- Не полагайтесь на мгновенное обновление через Volume (может занять до 60 секунд)

Пример структуры:

```
configs/
├── dev/
|   └── postgres-config.yaml
├── staging/
|   └── postgres-config.yaml
└── prod/
    └── postgres-config.yaml
```

1.3 Secrets: безопасное хранение паролей и ключей

1.3.1 Мотивация: проблема хранения паролей

Плохо:

```yaml
env:
- name: POSTGRES_PASSWORD
  value: "SuperSecret123!"  # ❌ Пароль в открытом виде в Git!
```

Проблемы:

- ❌ Пароль попадает в Git (утечка данных)
- ❌ Любой с доступом к репозиторию видит секреты
- ❌ Невозможно ротировать пароли без изменения манифестов
- ❌ Нарушение compliance (SOC2, ISO 27001, PCI DSS)

1.3.2 Решение: Secrets

Secret — специальный объект Kubernetes для хранения чувствительных данных (пароли, токены, SSH-ключи, TLS-сертификаты).

Отличия Secret от ConfigMap:

Характеристика	ConfigMap	Secret
Назначение	Конфигурация	Чувствительные данные
Кодирование	Открытый текст	Base64
Шифрование в etcd	❌ Нет (по умолчанию)	✅ Да (если включено)
Доступ через RBAC	Обычный	Строгий
Максимальный размер	1 MB	1 MB
Аудит доступа	Стандартный	Расширенный

⚠️ **Важно:** По умолчанию Secret хранится в etcd в виде Base64, что **НЕ является шифрованием**. Для настоящей безопасности нужно включить Encryption at Rest.

1.3.3 Создание Secret

Способ 1: Через kubectl (рекомендуется для локальной разработки)

```
# Generic secret из литералов
kubectl create secret generic postgres-secret \
  --from-literal=POSTGRES_PASSWORD=SuperSecret123! \
  --from-literal=POSTGRES_USER=postgres

# Из файла
echo -n "SuperSecret123!" > password.txt
kubectl create secret generic postgres-secret \
  --from-file=password=password.txt

# ⚠️ Не забудьте удалить файл!
rm password.txt

# Проверка
kubectl get secrets
kubectl describe secret postgres-secret  # Значения не показываются!
```

Способ 2: Через YAML (для production с внешним управлением секретами)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque  # Самый распространённый тип
data:
  # Значения в Base64
  POSTGRES_USER: cG9zdGdyZXM=  # echo -n "postgres" | base64
  POSTGRES_PASSWORD: U3VwZXJTZWNyZXQxMjMh  # echo -n "SuperSecret123!" | base64
```

⚠️ **Проблема:** Base64-значения легко декодируются:

```bash
echo "U3VwZXJTZWNyZXQxMjMh" | base64 -d
# SuperSecret123!
```

Решение: Используйте External Secrets Operator или Sealed Secrets (см. ниже).

Способ 3: stringData (удобнее для чтения)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  # Kubernetes сам закодирует в Base64
  POSTGRES_USER: "postgres"
  POSTGRES_PASSWORD: "SuperSecret123!"
```

⚠️ **Внимание:** Даже с `stringData` секрет попадёт в Git в открытом виде!

1.3.4 Использование Secrets в Pod

Способ 1: Environment Variables

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    env:
    - name: POSTGRES_USER
      valueFrom:
        secretKeyRef:
          name: postgres-secret
          key: POSTGRES_USER
    - name: POSTGRES_PASSWORD
      valueFrom:
        secretKeyRef:
          name: postgres-secret
          key: POSTGRES_PASSWORD
```

Способ 2: Volume (файлы)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    volumeMounts:
    - name: secret-volume
      mountPath: /run/secrets
      readOnly: true  # ✅ Важно для безопасности!
  volumes:
  - name: secret-volume
    secret:
      secretName: postgres-secret
      defaultMode: 0400  # Только чтение для владельца
```

Внутри контейнера:

```
ls -la /run/secrets/
# -r-------- 1 root root  8 Feb 13 10:00 POSTGRES_USER
# -r-------- 1 root root 15 Feb 13 10:00 POSTGRES_PASSWORD

cat /run/secrets/POSTGRES_PASSWORD
# SuperSecret123!
```

1.3.5 Специальные типы Secrets

1. Docker Registry Secret (для приватных образов)

```
kubectl create secret docker-registry my-registry-secret \
  --docker-server=registry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword \
  --docker-email=user@example.com
```

Использование:

```
spec:
  imagePullSecrets:
  - name: my-registry-secret
  containers:
  - name: postgres
    image: registry.example.com/postgres:16-custom
```

2. TLS Secret (для SSL-сертификатов)

```
kubectl create secret tls postgres-tls-secret \
  --cert=server.crt \
  --key=server.key
```

Использование для PostgreSQL SSL:

```yaml
volumeMounts:
- name: tls-volume
  mountPath: /var/lib/postgresql/ssl
volumes:
- name: tls-volume
  secret:
    secretName: postgres-tls-secret
    items:
    - key: tls.crt
      path: server.crt
      mode: 0600
    - key: tls.key
      path: server.key
      mode: 0600
```

1.3.6 Безопасное управление Secrets в Production

Проблема: Нельзя хранить Secrets в Git в открытом виде.

Решение 1: Sealed Secrets (Bitnami)

```bash
# Установка контроллера
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/
        download/v0.24.0/controller.yaml

# Установка CLI
brew install kubeseal

# Создать обычный Secret
kubectl create secret generic postgres-secret \
  --from-literal=POSTGRES_PASSWORD=SuperSecret123! \
  --dry-run=client -o yaml > secret.yaml

# Зашифровать
kubeseal --format yaml < secret.yaml > sealed-secret.yaml

# Можно коммитить в Git!
git add sealed-secret.yaml
```

sealed-secret.yaml:

```yaml
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: postgres-secret
spec:
  encryptedData:
    POSTGRES_PASSWORD: AgB7zX... (зашифровано!)
```

Применение:

```bash
kubectl apply -f sealed-secret.yaml
# Контроллер автоматически создаст обычный Secret
```

Решение 2: External Secrets Operator

Интеграция с AWS Secrets Manager, Azure Key Vault, HashiCorp Vault, Google Secret Manager.

```yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: postgres-secret
spec:
  secretStoreRef:
    name: aws-secrets-manager  # Или vault, gcpsm, etc.
    kind: SecretStore
  target:
    name: postgres-secret  # Имя создаваемого Secret
  data:
  - secretKey: POSTGRES_PASSWORD
    remoteRef:
      key: prod/postgres/password  # Путь в AWS Secrets Manager
```

Преимущества:

- ✅ Секреты хранятся в специализированных системах
- ✅ Централизованная ротация паролей
- ✅ Аудит доступа
- ✅ Автоматическая синхронизация

Решение 3: HashiCorp Vault

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "postgres-role"
    vault.hashicorp.com/agent-inject-secret-password: "database/creds/postgres"
spec:
  serviceAccountName: postgres-sa
  containers:
  - name: postgres
    image: postgres:16
    command:
    - sh
    - -c
    - |
      export POSTGRES_PASSWORD=$(cat /vault/secrets/password)
      docker-entrypoint.sh postgres
```

1.3.7 Ротация Secrets

Сценарий: Нужно изменить пароль PostgreSQL.

Шаг 1: Обновить Secret

```bash
kubectl delete secret postgres-secret
kubectl create secret generic postgres-secret \
  --from-literal=POSTGRES_PASSWORD=NewPassword456!
```

Шаг 2: Перезапустить поды

```bash
kubectl rollout restart deployment postgres-deployment
```

Шаг 3: Обновить пароль в PostgreSQL (если база уже создана)

```bash
kubectl exec -it postgres-pod -- psql -U postgres -c "ALTER USER postgres WITH
        PASSWORD 'NewPassword456!';"
```

Автоматическая ротация:

- Используйте External Secrets Operator с автоматической ротацией
- Настройте Kubernetes CronJob для периодической ротации

1.3.8 Best Practices для Secrets

✅ **Делайте:**

- Используйте RBAC для ограничения доступа к Secrets
- Включите Encryption at Rest в etcd
- Используйте External Secrets Operator или Sealed Secrets для production
- Монтируйте Secrets как Volume (безопаснее, чем env vars)
- Устанавливайте `readOnly: true` для Secret volumes
- Регулярно ротируйте пароли

❌ **Не делайте:**

- Не храните Secrets в Git в открытом виде
- Не используйте ConfigMap для паролей
- Не логируйте значения Secrets
- Не используйте один Secret для разных окружений

1.4 Volumes: основы хранения данных

1.4.1 Мотивация: проблема эфемерности контейнеров

По умолчанию контейнеры эфемерны:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    env:
    - name: POSTGRES_PASSWORD
      value: demo123
```

Что произойдёт:

```
# Создать таблицу
kubectl exec -it postgres-pod -- psql -U postgres -c "CREATE TABLE users (id
        SERIAL, name TEXT);"
kubectl exec -it postgres-pod -- psql -U postgres -c "INSERT INTO users (name)
        VALUES ('Alice');"

# Проверить
kubectl exec -it postgres-pod -- psql -U postgres -c "SELECT * FROM users;"
#  id | name
# ----+-------
#   1 | Alice

# ⚠️ Перезапустить под
kubectl delete pod postgres-pod
kubectl apply -f postgres-pod.yaml

# 💥 Данные потеряны!
kubectl exec -it postgres-pod -- psql -U postgres -c "SELECT * FROM users;"
# ERROR:  relation "users" does not exist
```

Проблема: Файловая система контейнера удаляется при перезапуске пода.

1.4.2 Решение: Volumes

Volume — это директория, доступная контейнерам в поде. Данные в Volume могут переживать перезапуск контейнера (но не всегда пода).

1.4.3 Типы Volumes

1. emptyDir (временное хранилище)

Создаётся при создании пода, удаляется при удалении пода.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    volumeMounts:
    - name: pgdata
      mountPath: /var/lib/postgresql/data
  volumes:
  - name: pgdata
    emptyDir: {}  # Пустой том
```

Использование:

- ✅ Временные файлы
- ✅ Кэш
- ✅ Обмен данными между контейнерами в одном поде
- ❌ **НЕ для production баз данных!**

2. hostPath (монтирование с хоста)

Монтирует директорию с ноды Kubernetes.

```
volumes:
- name: pgdata
  hostPath:
    path: /mnt/data/postgres
    type: DirectoryOrCreate
```

⚠️ **Проблемы:**

- ❌ Данные привязаны к конкретной ноде
- ❌ Если под переедет на другую ноду — данные потеряны
- ❌ Нарушение портабельности
- ❌ Проблемы безопасности (доступ к файловой системе хоста)

Использование:

- ⚠️ Только для тестирования на одноузловых кластерах
- ⚠️ Для системных агентов (node exporter, log collectors)
- ❌ **НЕ для production баз данных!**

3. Persistent Volumes (правильное решение для баз данных)

См. следующий раздел.

1.5 Persistent Volumes: долговечное хранилище

1.5.1 Архитектура PV и PVC

PersistentVolume (PV) — физический том хранилища (диск). **PersistentVolumeClaim (PVC)** — запрос на том определённого размера и типа.

Аналогия для DBA:

- **PV** — это как физический жёсткий диск в сервере
- **PVC** — это как заявка системному администратору: “Мне нужен диск 100GB SSD”
- **StorageClass** — это как типы дисков: SSD, HDD, NVMe

Схема взаимодействия:

```
Pod → PVC (запрос) → PV (физический диск) → Cloud Storage (AWS EBS, Azure Disk,
GCP PD)
```

1.5.2 Создание Persistent Volume

Ручное создание PV (статическое выделение):

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: postgres-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
  - ReadWriteOnce  # RWO: только одна нода может писать
  persistentVolumeReclaimPolicy: Retain  # Что делать при удалении PVC
  storageClassName: manual
  hostPath:
    path: /mnt/data/postgres  # ⚠️ Только для тестирования!
```

Создание PVC (запрос на том):

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: manual
```

Применение:

```
kubectl apply -f postgres-pv.yaml
kubectl apply -f postgres-pvc.yaml

# Проверка
kubectl get pv
# NAME          CAPACITY   ACCESS MODES   STATUS   CLAIM
        STORAGECLASS
# postgres-pv   10Gi       RWO            Bound    default/postgres-pvc
        manual

kubectl get pvc
# NAME           STATUS   VOLUME        CAPACITY   ACCESS MODES
# postgres-pvc   Bound    postgres-pv   10Gi       RWO
```

Использование PVC в Pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
  - name: postgres
    image: postgres:16
    env:
    - name: POSTGRES_PASSWORD
      valueFrom:
        secretKeyRef:
          name: postgres-secret
          key: POSTGRES_PASSWORD
    - name: PGDATA
      value: /var/lib/postgresql/data/pgdata  # ⚠️ Важно!
    volumeMounts:
    - name: pgdata
      mountPath: /var/lib/postgresql/data
  volumes:
  - name: pgdata
    persistentVolumeClaim:
      claimName: postgres-pvc
```

⚠️ PGDATA важность:

PostgreSQL требует, чтобы PGDATA была подкаталогом монтированного тома, иначе:

```
initdb: error: directory "/var/lib/postgresql/data" exists but is not empty
```

1.5.3 Access Modes

Mode	Сокращение	Описание	Использование
ReadWriteOnce	RWO	Чтение-запись одной нодой	PostgreSQL Primary, MySQL
ReadOnlyMany	ROX	Только чтение многими нодами	Статические файлы, конфиги
ReadWriteMany	RWX	Чтение-запись многими нодами	Shared storage (NFS, CephFS)
ReadWriteOncePod	RWOP	Чтение-запись одним подом	Гарантия эксклюзивности

Для PostgreSQL:

- **Primary:** RWO (только одна нода пишет)
- **Replicas:** Каждая реплика должна иметь свой RWO том
- **Shared configs:** ROX

1.5.4 Reclaim Policy

Что происходит с PV при удалении PVC:

Политика	Описание	Использование
Retain	PV сохраняется, данные не удаляются	Production (ручная очистка)
Delete	PV и данные удаляются автоматически	Development, временные окружения
Recycle	⚠️ Устарело	Не использовать

Пример:

```
spec:
  persistentVolumeReclaimPolicy: Retain  # Для production
```

Сценарий с Retain:

```
# Удалить PVC
kubectl delete pvc postgres-pvc

# PV переходит в статус Released (но данные сохранены)
kubectl get pv
# NAME          STATUS     CLAIM
# postgres-pv   Released   default/postgres-pvc

# Для переиспользования нужно вручную очистить claimRef
kubectl patch pv postgres-pv -p '{"spec":{"claimRef":null}}'

# Теперь PV снова Available
kubectl get pv
# NAME          STATUS
# postgres-pv   Available
```

1.6 StatefulSets: для stateful приложений

1.6.1 Почему Deployment не подходит для баз данных

Проблемы Deployment для PostgreSQL:

1. **Случайные имена подов:** `postgres-deployment-7d9f8c6b5-x4k2p`
   - Невозможно предсказать имя
   - DNS-имя меняется при пересоздании
2. **Случайный порядок запуска/остановки**
   - Поды создаются параллельно
   - Нет гарантии, что Primary запустится раньше Replica
3. **Один PVC для всех подов**
   - Все реплики пишут в один том (конфликт!)
   - Или каждый под получает emptyDir (данные теряются)

1.6.2 Решение: StatefulSet

StatefulSet — контроллер для stateful приложений, обеспечивающий:

- ✅ Стабильные имена подов: `postgres-0`, `postgres-1`, `postgres-2`
- ✅ Стабильные DNS-имена: `postgres-0.postgres-headless.default.svc.cluster.local`
- ✅ Упорядоченный запуск: 0 → 1 → 2
- ✅ Упорядоченная остановка: 2 → 1 → 0

- ✅ Индивидуальные PVC для каждого пода

1.6.3 Пример StatefulSet для PostgreSQL

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None  # Headless Service для StatefulSet
  selector:
    app: postgres
  ports:
  - port: 5432
    targetPort: 5432

---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless  # Обязательно!
  replicas: 3  # 1 Primary + 2 Replicas (нужна настройка репликации отдельно)
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:16
        ports:
        - containerPort: 5432
          name: postgres
        env:
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: POSTGRES_PASSWORD
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
        volumeMounts:
        - name: pgdata
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:  # Автоматическое создание PVC для каждого пода
  - metadata:
      name: pgdata
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: standard  # Используем StorageClass
```

```
      resources:
        requests:
          storage: 10Gi
```

Применение:

```
kubectl apply -f postgres-statefulset.yaml

# Проверка
kubectl get statefulset
kubectl get pods
# NAME         READY   STATUS    RESTARTS   AGE
# postgres-0   1/1     Running   0          2m
# postgres-1   1/1     Running   0          90s
# postgres-2   1/1     Running   0          60s

# PVC созданы автоматически
kubectl get pvc
# NAME                STATUS   VOLUME
        CAPACITY
# pgdata-postgres-0  Bound    pvc-abc123...                              10Gi
# pgdata-postgres-1  Bound    pvc-def456...                              10Gi
# pgdata-postgres-2  Bound    pvc-ghi789...                              10Gi
```

1.6.4 DNS-имена для StatefulSet

Формат:

```
<pod-name>.<service-name>.<namespace>.svc.cluster.local
```

Примеры:

```
# Внутри кластера
psql -h postgres-0.postgres-headless.default.svc.cluster.local -U postgres

# Или короче
psql -h postgres-0.postgres-headless -U postgres

# Из того же namespace
psql -h postgres-0.postgres-headless -U postgres
```

1.6.5 Упорядоченный запуск и остановка

Запуск:

```
kubectl scale statefulset postgres --replicas=3

# Логи:
# Creating pod postgres-0... Done
# Waiting for postgres-0 to be Ready...
# Creating pod postgres-1... Done
# Waiting for postgres-1 to be Ready...
# Creating pod postgres-2... Done
```

Остановка:

```
kubectl scale statefulset postgres --replicas=0

# Логи:
# Terminating pod postgres-2... Done
# Terminating pod postgres-1... Done
# Terminating pod postgres-0... Done
```

⚠️ Важно для PostgreSQL:

- Primary должен быть `postgres-0`
- Replicas — `postgres-1`, `postgres-2`, …
- При остановке реплики останавливаются первыми

1.6.6 Update Strategy

RollingUpdate (по умолчанию):

```yaml
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0  # Обновлять с этого индекса
```

Обновление с partition:

```
# Обновить только postgres-2 (для тестирования)
kubectl patch statefulset postgres -p '{"spec":{"updateStrategy":
        {"rollingUpdate":{"partition":2}}}}'

# Обновить образ
kubectl set image statefulset/postgres postgres=postgres:16.1

# Только postgres-2 обновится
# После проверки: обновить всех
kubectl patch statefulset postgres -p '{"spec":{"updateStrategy":
        {"rollingUpdate":{"partition":0}}}}'
```

OnDelete:

```yaml
spec:
  updateStrategy:
    type: OnDelete
```

Поды обновляются только при ручном удалении:

```
kubectl delete pod postgres-2
# Пересоздастся с новым образом
```

1.7 Storage Classes: динамическое выделение хранилища

1.7.1 Проблема статического выделения

Ручное создание PV:

- ❌ Нужно заранее создавать PV для каждого приложения
- ❌ Сложно масштабировать (100 баз = 100 PV вручную)
- ❌ Невозможно автоматически выделить диск в облаке

1.7.2 Решение: StorageClass

StorageClass — это шаблон для автоматического создания PV при создании PVC.

Провайдеры StorageClass:

Платформа	Provisioner	Тип диска
AWS	`ebs.csi.aws.com`	EBS (Elastic Block Store)
Azure	`disk.csi.azure.com`	Azure Managed Disk
GCP	`pd.csi.storage.gke.io`	Persistent Disk
Local	`kubernetes.io/no-provisioner`	Локальные диски ноды

1.7.3 Пример: AWS EBS StorageClass

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3  # General Purpose SSD v3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
  kmsKeyId: arn:aws:kms:us-east-1:123456789:key/abc-def-ghi  # Опционально
volumeBindingMode: WaitForFirstConsumer  # Создать диск в той же зоне, что и
        под
allowVolumeExpansion: true  # Разрешить расширение диска
reclaimPolicy: Delete
```

Использование:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: fast-ssd  # Используем StorageClass
  resources:
    requests:
      storage: 100Gi
```

Что произойдёт:

```
kubectl apply -f postgres-pvc.yaml

# 1. PVC создан, но PV ещё нет (WaitForFirstConsumer)
kubectl get pvc
# NAME           STATUS    VOLUME
# postgres-pvc   Pending

# 2. Создаём под, который использует PVC
kubectl apply -f postgres-statefulset.yaml

# 3. Kubernetes автоматически создаёт EBS диск в AWS
# 4. PV создаётся и привязывается к PVC
kubectl get pv
# NAME                                       CAPACITY   ACCESS MODES
        STORAGECLASS
# pvc-abc123-456def-789ghi                  100Gi      RWO            fast-ssd

kubectl get pvc
# NAME           STATUS   VOLUME                      CAPACITY
# postgres-pvc   Bound    pvc-abc123-456def-789ghi   100Gi
```

1.7.4 Volume Binding Modes

Immediate (по умолчанию):

PV создаётся сразу при создании PVC, **до создания пода**.

Проблема:

- Диск создаётся в случайной availability zone
- Под может быть запланирован в другой зоне → не может примонтировать диск

WaitForFirstConsumer (рекомендуется):

PV создаётся только после создания пода, в той же зоне.

```
volumeBindingMode: WaitForFirstConsumer
```

1.7.5 Расширение дисков (Volume Expansion)

Включить в StorageClass:

```
allowVolumeExpansion: true
```

Расширить PVC:

```
# Редактировать PVC
kubectl edit pvc postgres-pvc

# Изменить размер
spec:
  resources:
    requests:
      storage: 200Gi  # Было 100Gi
```

Что происходит:

1. Kubernetes изменяет размер диска в облаке (AWS EBS resize)
2. Перезапускает под (для применения изменений)
3. Файловая система расширяется автоматически (для ext4, xfs)

⚠️ **Ограничения:**

- ❌ Нельзя уменьшить размер диска
- ⚠️ Требуется перезапуск пода (downtime)

Для PostgreSQL без downtime:

- Используйте репликацию (failover на реплику во время расширения Primary)
- Или используйте PostgreSQL Operator (автоматическое управление)

1.7.6 Снапшоты (VolumeSnapshot)

VolumeSnapshot — создание снимка PVC для бэкапа.

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: postgres-snapshot
spec:
  volumeSnapshotClassName: csi-snapshot-class
  source:
    persistentVolumeClaimName: postgres-pvc
```

Создание:

```
kubectl apply -f postgres-snapshot.yaml

# Проверка
kubectl get volumesnapshot
# NAME                READYTOUSE   SOURCEPVC      RESTORETIME   AGE
# postgres-snapshot   true         postgres-pvc   12s           1m
```

Восстановление из снапшота:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc-restored
spec:
  dataSource:
    name: postgres-snapshot
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 100Gi
```

1.8 Практика: PostgreSQL с PVC

1.8.1 Полный пример: PostgreSQL StatefulSet в AWS EKS

1. Создать Secret для пароля:

```yaml
# postgres-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "ProductionPassword123!"
  POSTGRES_USER: "postgres"
```

2. Создать StorageClass:

```yaml
# postgres-storageclass.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: postgres-storage
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Retain  # ⚠️ Не удалять данные при удалении PVC!
```

3. Создать ConfigMap для конфигурации:

```yaml
# postgres-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-config
data:
  POSTGRES_DB: "production_db"

  custom.conf: |
    # Performance tuning
    shared_buffers = 256MB
    effective_cache_size = 1GB
    maintenance_work_mem = 64MB
    checkpoint_completion_target = 0.9
    wal_buffers = 16MB
    default_statistics_target = 100
    random_page_cost = 1.1
    effective_io_concurrency = 200
    work_mem = 4MB
    min_wal_size = 1GB
    max_wal_size = 4GB

    # Logging
    log_min_duration_statement = 1000
    log_connections = on
    log_disconnections = on
    log_duration = off
    log_statement = 'ddl'

    # Extensions
    shared_preload_libraries = 'pg_stat_statements'
```

4. Создать Headless Service:

```yaml
# postgres-headless-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
  labels:
    app: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
  - port: 5432
    targetPort: 5432
    name: postgres
```

5. Создать StatefulSet:

```yaml
# postgres-statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless
  replicas: 1  # Primary only (для начала)
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:16
        ports:
        - containerPort: 5432
          name: postgres

        env:
        # Секреты
        - name: POSTGRES_USER
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: POSTGRES_USER
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: POSTGRES_PASSWORD

        # Конфигурация из ConfigMap
        - name: POSTGRES_DB
          valueFrom:
            configMapKeyRef:
              name: postgres-config
              key: POSTGRES_DB

        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata

        volumeMounts:
        - name: pgdata
          mountPath: /var/lib/postgresql/data

        - name: config
```

```yaml
          mountPath: /etc/postgresql/custom.conf
          subPath: custom.conf

        # Resources (важно для production!)
        resources:
          requests:
            memory: "1Gi"
            cpu: "500m"
          limits:
            memory: "2Gi"
            cpu: "1000m"

        # Liveness probe
        livenessProbe:
          exec:
            command:
            - sh
            - -c
            - pg_isready -U $POSTGRES_USER
          initialDelaySeconds: 30
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 3

        # Readiness probe
        readinessProbe:
          exec:
            command:
            - sh
            - -c
            - pg_isready -U $POSTGRES_USER
          initialDelaySeconds: 5
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 3

      volumes:
      - name: config
        configMap:
          name: postgres-config

  volumeClaimTemplates:
  - metadata:
      name: pgdata
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: postgres-storage
      resources:
        requests:
          storage: 50Gi
```

6. Создать обычный Service для доступа (опционально):

```yaml
# postgres-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  labels:
    app: postgres
spec:
  type: ClusterIP
  selector:
    app: postgres
    # ⚠️ Для мультиреплик нужен selector по роли (primary/replica)
  ports:
  - port: 5432
    targetPort: 5432
    name: postgres
```

Применение:

```bash
# В правильном порядке
kubectl apply -f postgres-secret.yaml
kubectl apply -f postgres-storageclass.yaml
kubectl apply -f postgres-config.yaml
kubectl apply -f postgres-headless-service.yaml
kubectl apply -f postgres-statefulset.yaml
kubectl apply -f postgres-service.yaml

# Проверка
kubectl get all
kubectl get pvc
kubectl get pv
```

Подключение к PostgreSQL:

```bash
# Из пода в том же namespace
psql -h postgres.default.svc.cluster.local -U postgres -d production_db

# Или через port-forward (для локального доступа)
kubectl port-forward postgres-0 5432:5432

# На локальной машине
psql -h localhost -U postgres -d production_db
```

Проверка персистентности:

```
# Создать таблицу
kubectl exec -it postgres-0 -- psql -U postgres -d production_db -c "CREATE
        TABLE test (id SERIAL, data TEXT);"
kubectl exec -it postgres-0 -- psql -U postgres -d production_db -c "INSERT
        INTO test (data) VALUES ('persistent data');"

# Удалить под
kubectl delete pod postgres-0

# StatefulSet пересоздаст под
kubectl get pods -w

# Проверить данные
kubectl exec -it postgres-0 -- psql -U postgres -d production_db -c "SELECT *
        FROM test;"
#  id |      data
# ----+----------------
#   1 | persistent data
# ✅ Данные сохранены!
```

1.9 Helm: пакетный менеджер Kubernetes

1.9.1 Зачем нужен Helm

Проблема: Управление множеством YAML-файлов.

Для PostgreSQL мы создали: - Secret - ConfigMap - StorageClass - Service (headless) - Service (обычный) - StatefulSet

Всего: 6 файлов. Для production окружения может быть 20+ файлов.

Проблемы:

- ❌ Сложно управлять версиями
- ❌ Нужно дублировать для dev/staging/prod
- ❌ Трудно обновлять (какие файлы изменились?)
- ❌ Нет зависимостей между ресурсами

1.9.2 Helm: решение

Helm — пакетный менеджер для Kubernetes (как apt/yum для Linux, npm для Node.js).

Chart — пакет Helm, содержащий шаблоны Kubernetes-ресурсов.

Преимущества:

- ✅ Один Chart = много ресурсов
- ✅ Параметризация через `values.yaml`
- ✅ Версионирование релизов
- ✅ Откат к предыдущим версиям
- ✅ Готовые Charts из Artifact Hub

1.9.3 Установка Helm

```
# MacOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Проверка
helm version
# version.BuildInfo{Version:"v3.14.0", ...}
```

1.9.4 Использование готовых Charts

Пример: PostgreSQL от Bitnami

```
# Добавить репозиторий Bitnami
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Поиск Chart
helm search repo postgresql
# NAME                    CHART VERSION   APP VERSION DESCRIPTION
# bitnami/postgresql      14.0.5          16.1.0      PostgreSQL is an advanced
        object-relational...

# Просмотр значений по умолчанию
helm show values bitnami/postgresql > values.yaml
```

Файл: `values.yaml` **(кастомизация)**

```yaml
# values.yaml
global:
  postgresql:
    auth:
      username: "postgres"
      password: "ProductionPassword123!"  # ⚠️ Лучше через --set или external
        secrets
      database: "production_db"

image:
  tag: "16.1.0"

primary:
  resources:
    requests:
      memory: "1Gi"
      cpu: "500m"
    limits:
      memory: "2Gi"
      cpu: "1000m"

  persistence:
    enabled: true
    storageClass: "postgres-storage"
    size: 50Gi

readReplicas:
  replicaCount: 2
  resources:
    requests:
      memory: "512Mi"
      cpu: "250m"
    limits:
      memory: "1Gi"
      cpu: "500m"

metrics:
  enabled: true  # Включить postgres_exporter для Prometheus
```

Установка:

```
# Установить Chart
helm install my-postgres bitnami/postgresql -f values.yaml

# Или без файла (через --set)
helm install my-postgres bitnami/postgresql \
  --set auth.password=ProductionPassword123! \
  --set primary.persistence.size=50Gi

# Проверка
helm list
# NAME          NAMESPACE   REVISION    UPDATED
        STATUS      CHART               APP VERSION
# my-postgres   default     1           2024-02-13 10:00:00.123456 +0300 MSK
        deployed    postgresql-14.0.5   16.1.0

kubectl get all
# StatefulSet, Services, Pods созданы автоматически!
```

Получение пароля:

```
export POSTGRES_PASSWORD=$(kubectl get secret --namespace default my-postgres-
        postgresql -o jsonpath="{.data.postgres-password}" | base64 -d)

echo $POSTGRES_PASSWORD
```

Подключение:

```
kubectl port-forward --namespace default svc/my-postgres-postgresql 5432:5432 &

PGPASSWORD="$POSTGRES_PASSWORD" psql -h 127.0.0.1 -U postgres -d production_db
```

Обновление релиза:

```
# Изменить values.yaml
# Например, увеличить replicas

# Применить изменения
helm upgrade my-postgres bitnami/postgresql -f values.yaml

# История версий
helm history my-postgres

# Откат к предыдущей версии
helm rollback my-postgres 1
```

Удаление:

```
helm uninstall my-postgres

# ⚠️ PVC не удаляются автоматически (защита данных)!
kubectl get pvc
kubectl delete pvc data-my-postgres-postgresql-0
```

1.9.5 Создание своего Chart

```
helm create my-postgres-chart

# Структура:
# my-postgres-chart/
# ├── Chart.yaml        # Метаданные Chart
# ├── values.yaml       # Значения по умолчанию
# ├── templates/        # Шаблоны Kubernetes-ресурсов
# |   ├── deployment.yaml
# |   ├── service.yaml
# |   └── ...
# └── charts/           # Зависимости (sub-charts)
```

Chart.yaml:

```yaml
apiVersion: v2
name: my-postgres-chart
description: PostgreSQL Helm Chart for Production
type: application
version: 1.0.0
appVersion: "16.1.0"
```

values.yaml:

```yaml
image:
  repository: postgres
  tag: "16"
  pullPolicy: IfNotPresent

auth:
  username: postgres
  password: ""  # Передавать через --set
  database: myapp

persistence:
  enabled: true
  storageClass: "postgres-storage"
  size: 50Gi

resources:
  requests:
    memory: "1Gi"
    cpu: "500m"
  limits:
    memory: "2Gi"
    cpu: "1000m"
```

templates/statefulset.yaml:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: {{ include "my-postgres-chart.fullname" . }}
  labels:
    {{- include "my-postgres-chart.labels" . | nindent 4 }}
spec:
  serviceName: {{ include "my-postgres-chart.fullname" . }}-headless
  replicas: 1
  selector:
    matchLabels:
      {{- include "my-postgres-chart.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "my-postgres-chart.selectorLabels" . | nindent 8 }}
    spec:
      containers:
      - name: postgres
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        env:
        - name: POSTGRES_USER
          value: {{ .Values.auth.username }}
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: {{ include "my-postgres-chart.fullname" . }}-secret
              key: password
        - name: POSTGRES_DB
          value: {{ .Values.auth.database }}
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
        ports:
        - containerPort: 5432
          name: postgres
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
        resources:
          {{- toYaml .Values.resources | nindent 10 }}
  {{- if .Values.persistence.enabled }}
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: {{ .Values.persistence.storageClass }}
      resources:
        requests:
```

```
            storage: {{ .Values.persistence.size }}
  {{- end }}
```

Установка своего Chart:

```
helm install my-postgres ./my-postgres-chart \
  --set auth.password=SecretPassword123!
```

1.10 PostgreSQL Operators: автоматизация управления

1.10.1 Проблема ручного управления

Даже с Helm и StatefulSet вам нужно:

- ❌ Вручную настраивать репликацию (Primary/Replica)
- ❌ Вручную делать failover при отказе Primary
- ❌ Вручную управлять бэкапами
- ❌ Вручную обновлять PostgreSQL
- ❌ Вручную масштабировать реплики

1.10.2 Решение: Kubernetes Operators

Operator — это расширение Kubernetes, которое использует custom resources для управления сложными приложениями.

Аналогия для DBA:

Operator — это как автоматизированный DBA-робот:

- Следит за состоянием кластера PostgreSQL
- Автоматически восстанавливает после сбоев
- Делает бэкапы по расписанию
- Обновляет версии PostgreSQL
- Масштабирует реплики

1.10.3 Три главных PostgreSQL Operator

1. CloudNativePG (ранее EDB Postgres for Kubernetes)

Разработчик: EnterpriseDB (EDB) **Статус:** CNCF Sandbox project

Особенности:

- ✅ Самый современный (активная разработка)
- ✅ Declarative API (всё через YAML)
- ✅ Встроенные бэкапы (Barman)
- ✅ Автоматический failover (через Patroni-like логику)
- ✅ Pooling через PgBouncer (встроенный)
- ✅ Мониторинг (Prometheus metrics)

Установка:

```
kubectl apply -f https://raw.githubusercontent.com/cloudnative-pg/cloudnative-
        pg/release-1.22/releases/cnpg-1.22.0.yaml

# Проверка
kubectl get deployment -n cnpg-system
```

Создание кластера PostgreSQL:

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-cluster
spec:
  instances: 3  # 1 Primary + 2 Replicas

  postgresql:
    parameters:
      shared_buffers: "256MB"
      max_connections: "200"
      log_min_duration_statement: "1000"

  bootstrap:
    initdb:
      database: production_db
      owner: app_user
      secret:
        name: app-user-secret

  storage:
    size: 50Gi
    storageClass: postgres-storage

  backup:
    barmanObjectStore:
      destinationPath: s3://my-backups/postgres-cluster
      s3Credentials:
        accessKeyId:
          name: aws-creds
          key: ACCESS_KEY_ID
        secretAccessKey:
          name: aws-creds
          key: SECRET_ACCESS_KEY
      wal:
        compression: gzip
    retentionPolicy: "30d"

  monitoring:
    enablePodMonitor: true
```

Применение:

```
kubectl apply -f postgres-cluster.yaml

# CloudNativePG автоматически создаёт:
# - StatefulSet с 3 подами
# - Services (rw, ro)
# - PVCs для каждого пода
# - ConfigMaps для конфигурации
# - Настраивает репликацию
# - Запускает бэкапы

kubectl get cluster
# NAME              AGE   INSTANCES   READY   STATUS
# postgres-cluster  2m    3           3       Cluster in healthy state

kubectl get pods
# NAME                 READY   STATUS    ROLE
# postgres-cluster-1   1/1     Running   Primary
# postgres-cluster-2   1/1     Running   Replica
# postgres-cluster-3   1/1     Running   Replica
```

Подключение:

```
# Primary (read-write)
psql -h postgres-cluster-rw -U app_user production_db

# Replicas (read-only)
psql -h postgres-cluster-ro -U app_user production_db
```

Симуляция отказа Primary:

```
# Удалить Primary
kubectl delete pod postgres-cluster-1

# CloudNativePG автоматически:
# 1. Делает promote одной из реплик → новый Primary
# 2. Пересоздаёт удалённый под → становится Replica
# 3. Переключает Service на новый Primary

# Всё происходит за ~30 секунд!
```

Бэкап и восстановление:

```
# Создать бэкап вручную
kubectl apply -f - <<EOF
apiVersion: postgresql.cnpg.io/v1
kind: Backup
metadata:
  name: postgres-backup-1
spec:
  cluster:
    name: postgres-cluster
EOF

# Проверка
kubectl get backup
# NAME                STATUS      AGE
# postgres-backup-1   completed   2m

# Восстановление из бэкапа
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-restored
spec:
  instances: 3

  bootstrap:
    recovery:
      backup:
        name: postgres-backup-1

  storage:
    size: 50Gi
```

2. Zalando Postgres Operator

Разработчик: Zalando (E-commerce, Германия) **Статус:** Production-ready, используется в Zalando с 2017 года

Особенности:

- ✅ Зрелый оператор (5+ лет в production)
- ✅ Patroni под капотом (HA через etcd/Consul)
- ✅ Connection pooling (PgBouncer)
- ✅ Управление логическими бэкапами (WAL-E, WAL-G)

Установка:

```
helm repo add postgres-operator-charts https://opensource.zalando.com/postgres-
        operator/charts/postgres-operator

helm install postgres-operator postgres-operator-charts/postgres-operator
```

Создание кластера:

```yaml
apiVersion: "acid.zalan.do/v1"
kind: postgresql
metadata:
  name: postgres-cluster
spec:
  teamId: "dba-team"
  volume:
    size: 50Gi
    storageClass: postgres-storage
  numberOfInstances: 3
  users:
    app_user:
    - superuser
    - createdb
  databases:
    production_db: app_user
  postgresql:
    version: "16"
    parameters:
      shared_buffers: "256MB"
      max_connections: "200"
  resources:
    requests:
      cpu: 500m
      memory: 1Gi
    limits:
      cpu: 1000m
      memory: 2Gi
```

3. Crunchy Postgres Operator (PGO)

Разработчик: Crunchy Data **Статус:** Certified for OpenShift

Особенности:

- ✅ Сертифицирован для Red Hat OpenShift
- ✅ Встроенный pgBackRest для бэкапов
- ✅ Поддержка PostGIS, TimescaleDB
- ✅ Monitoring через pgMonitor

1.10.4 Сравнение: Вручную vs Helm vs Operator

Задача	Вручную	Helm	Operator
Создание кластера	6+ YAML файлов	1 команда	1 YAML (6 строк)
Настройка репликации	Вручную (Patroni/repmgr)	Вручную	Автоматически
Failover	Вручную (`pg_ctl promote`)	Вручную	Автоматически (<30 сек)
Бэкапы	Cron + скрипты	Helm hooks	Автоматически (по расписанию)
Мониторинг	Вручную (Prometheus exporter)	Частично	Автоматически
Обновление PostgreSQL	Вручную (downtime)	Вручную	Rolling update (без downtime)
Масштабирование реплик	Вручную	`helm upgrade`	Изменить `instances: 5`

1.11 Troubleshooting и best practices

1.11.1 Проблема 1: Pod не может примонтировать PVC

Ошибка:

```
Events:
  Type     Reason              Message
  Warning  FailedAttachVolume  AttachVolume.Attach failed for volume "pvc-abc123" : rpc error: code = Internal desc = Could not attach volume "vol-xyz" to node "i-0123": attachment failed: volume is already attached to instance "i-0456"
```

Причина: PVC типа RWO привязан к другой ноде.

Решение:

```
# 1. Проверить, где запущен под
kubectl get pods -o wide

# 2. Проверить, к какой ноде привязан PV
kubectl get pv <pv-name> -o jsonpath='{.spec.nodeAffinity}'

# 3. Удалить под (Kubernetes переместит на правильную ноду)
kubectl delete pod <pod-name>

# Или использовать WaitForFirstConsumer в StorageClass
```

1.11.2 Проблема 2: PostgreSQL не запускается (PGDATA not empty)

Ошибка:

```
initdb: error: directory "/var/lib/postgresql/data" exists but is not empty
```

Причина: PVC содержит файлы от предыдущего запуска, но они не в подкаталоге.

Решение:

```
env:
- name: PGDATA
  value: /var/lib/postgresql/data/pgdata  # ✅ Подкаталог!
```

1.11.3 Проблема 3: Secret не обновляется в поде

Сценарий:

```
# Обновили Secret
kubectl create secret generic postgres-secret \
  --from-literal=POSTGRES_PASSWORD=NewPassword \
  --dry-run=client -o yaml | kubectl apply -f -

# Но под всё ещё использует старый пароль
```

Причина: Environment variables не обновляются автоматически.

Решение:

```
# Перезапустить поды
kubectl rollout restart statefulset postgres

# Или использовать Volume вместо env vars
```

1.11.4 Проблема 4: PVC застрял в статусе Pending

Проверка:

```
kubectl describe pvc postgres-pvc

# Events:
#   Warning  ProvisioningFailed  failed to provision volume: StorageClass
        "postgres-storage" not found
```

Причины:

1. **StorageClass не существует:**

```
kubectl get storageclass
# Создать StorageClass
```

1. **Нет provisioner:**

```
# Проверить, установлен ли CSI driver
kubectl get csidriver
```

1. **Недостаточно ресурсов в облаке:**

```
# AWS: проверить квоты EBS
aws service-quotas get-service-quota \
  --service-code ebs \
  --quota-code L-D18FCD1D
```

1.11.5 Проблема 5: Медленная работа базы данных

Диагностика:

```
# 1. Проверить IOPS диска
kubectl exec -it postgres-0 -- sh -c "dd if=/dev/zero of=/var/lib/postgresql/
        data/testfile bs=1M count=1024 oflag=direct"

# 2. Проверить настройки StorageClass
kubectl get storageclass postgres-storage -o yaml

# 3. Проверить метрики диска (AWS CloudWatch, Azure Monitor)
```

Решение:

- Увеличить IOPS в StorageClass (gp3: до 16000 IOPS)

- Использовать io2 вместо gp3 для критичных систем
- Включить provisioned IOPS

1.11.6 Best Practices

ConfigMaps и Secrets:

- ✅ Используйте External Secrets Operator или Sealed Secrets
- ✅ Не храните секреты в Git
- ✅ Версионируйте ConfigMaps (добавляйте суффикс с версией: `postgres-config-v2`)
- ✅ Используйте RBAC для ограничения доступа к Secrets

Volumes и Storage:

- ✅ Всегда используйте PVC для баз данных (НЕ emptyDir, НЕ hostPath)
- ✅ Используйте StorageClass с `volumeBindingMode: WaitForFirstConsumer`
- ✅ Используйте `reclaimPolicy: Retain` для production
- ✅ Делайте снапшоты PVC регулярно (VolumeSnapshot)
- ✅ Мониторьте использование дискового пространства

StatefulSets:

- ✅ Используйте StatefulSet для баз данных (НЕ Deployment)
- ✅ Настраивайте PodDisruptionBudget для защиты от случайного удаления
- ✅ Используйте Affinity/Anti-Affinity для распределения реплик по нодам
- ✅ Настраивайте Liveness и Readiness probes

PostgreSQL Operators:

- ✅ Используйте Operator для production (CloudNativePG, Zalando, Crunchy)
- ✅ Автоматизируйте бэкапы через Operator
- ✅ Тестируйте failover сценарии регулярно
- ✅ Мониторьте метрики через Prometheus

1.12 Полезные ресурсы

1.12.1 Официальная документация

Kubernetes:

- Kubernetes Storage

- StatefulSets
- ConfigMaps
- Secrets

PostgreSQL:

- PostgreSQL Docker Image
- PostgreSQL Configuration

1.12.2 PostgreSQL Operators

CloudNativePG:

- Документация
- GitHub
- Quickstart

Zalando Postgres Operator:

- Документация
- GitHub

Crunchy Postgres Operator:

- Документация
- GitHub

1.12.3 Helm Charts

- Bitnami PostgreSQL Chart
- Artifact Hub — поиск Helm Charts

1.12.4 Инструменты

Sealed Secrets:

- GitHub
- Документация

External Secrets Operator:

- GitHub
- Документация

Velero (бэкапы Kubernetes):

- GitHub
- Документация

1.12.5 Книги и курсы

- “Kubernetes in Action” by Marko Luksa (Manning, 2023)
- “Managing Kubernetes” by Brendan Burns, Craig Tracey (O’Reilly, 2023)
- “PostgreSQL: Up and Running” by Regina Obe, Leo Hsu (O’Reilly, 2024)

1.12.6 Блоги и статьи

- Kubernetes Blog
- CloudNativePG Blog
- Zalando Engineering Blog
- Percona Database Blog

1.13 Заключение

1.13.1 Что вы теперь знаете

После изучения этого дополнительного материала вы освоили:

✅ **ConfigMaps** — управление конфигурациями через декларативные манифесты, 3 способа использования

✅ **Secrets** — безопасное хранение паролей и ключей, интеграция с External Secrets, ротация

✅ **Volumes** — основы хранения данных, типы томов (emptyDir, hostPath, PV)

✅ **Persistent Volumes** — долговечное хранилище для баз данных, PV/PVC архитектура, Access Modes

✅ **StatefulSets** — управление stateful приложениями, стабильные имена, упорядоченный запуск

✅ **Storage Classes** — динамическое выделение дисков, интеграция с облачными провайдерами

✅ **Helm** — пакетный менеджер, использование готовых Charts, создание своих

✅ **PostgreSQL Operators** — автоматизация управления кластерами PostgreSQL, failover, бэкапы

✅ **Troubleshooting** — решение типичных проблем с PVC, Secrets, производительностью

1.13.2 Ключевые выводы

Для баз данных в Kubernetes:

1. **Никогда не используйте emptyDir или hostPath для production** — только PersistentVolumes
2. **Используйте StatefulSet, не Deployment** — для стабильных имён и индивидуальных PVC
3. **Храните пароли в Secrets, не в ConfigMaps** — используйте External Secrets для production
4. **Используйте PostgreSQL Operator** — автоматизация failover, бэкапов, обновлений
5. **Настраивайте мониторинг и алерты** — Prometheus, Grafana, pg_stat_statements
6. **Делайте регулярные бэкапы** — VolumeSnapshots, WAL archiving, логические бэкапы

1.13.3 Что дальше

На следующем занятии PostgreSQL Advanced курса:

Мы изучим Terraform для автоматизации развёртывания инфраструктуры PostgreSQL в облаках и управления конфигурациями через Infrastructure as Code.

Следующие шаги в вашем обучении:

1. **Практика:** Разверните PostgreSQL с PVC на локальном кластере (minikube, kind)
2. **Эксперименты:** Попробуйте CloudNativePG Operator и сравните с ручным управлением
3. **Автоматизация:** Настройте автоматические бэкапы через Operator
4. **Мониторинг:** Интегрируйте Prometheus и Grafana для мониторинга PostgreSQL
5. **Production readiness:** Настройте HA-кластер с автоматическим failover

---

Этот дополнительный материал подготовлен для курса “PostgreSQL Advanced” OTUS.
Автор: Дмитрий Золотов Дата: 2026-02-13

Удачи в освоении Kubernetes для PostgreSQL! 🚀